

Tracking Daily Inflation in Cambodia

The Plain-Language Guide to How We Measure Daily Prices, Clean Data, and Calculate the Consumer Price Index

What is this book about?

This handbook explains in simple, clear, everyday words how our automated system tracks prices across Cambodia every day. It shows how we collect over 45,000 prices every morning, fix typos, convert US Dollars to Khmer Riel, calculate fair inflation numbers, and predict where prices are heading weeks before official monthly reports come out.

No heavy technical jargon. Just clear explanations, practical everyday examples, and simple math.

Written by: Cambodia Inflation Intelligence Team

Target Country: Kingdom of Cambodia

Edition: Plain-Language Master Edition (September 2026)

Contents

Executive Summary: The Cambodia Daily CPI Architecture	3
1 Why Cambodia Needs a Daily Price Tracker	4
1.1 Inflation and the 30-Day Policy Blind Spot	4
1.2 The Reality: Prices Move in Hours, Not Months	4
1.3 The Solution: Modernizing Price Tracking with Data Disciplines	4
2 The 3-Step Journey of Our Data (Bronze, Silver, Gold)	5
2.1 The 3 Refining Steps (The Medallion Approach)	5
2.2 How Airflow and Cosmos Keep Everything Running on Time	5
2.3 Splitting Big Tables into Monthly Folders	5
3 Collecting the Prices Every Morning (Bronze Layer)	7
3.1 The Stores and Services We Actually Scrape	7
3.2 Master Inventory: Stores and Available Items	7
3.3 How We Scrape Each Source (Methods and Technology)	9
3.4 Visiting Websites Politely Without Getting Blocked	11
3.5 Never Recording the Same Price Twice (Deduplication)	12
3.6 The Ingestion Circuit Breaker and Dual-Currency Protection	12
4 Cleaning the Data and Text Normalization (Silver Layer)	13
4.1 The Khmer and English Language Challenge	13
4.2 Text Cleaning and Unit Normalization Rules	13
5 AI Vector Embeddings: Matching Items and Sorting Products	14
5.1 What is a Vector Embedding?	14
5.2 Part 1: Semantic Product Matching (Connecting Scraped Names to Catalog)	14
5.2.1 The Mathematical Cosine Similarity	14
5.2.2 The 3 Matching Tiers and Decision Rules	14
5.2.3 Crucial Spec Guardrails (Preventing False Merges)	14
5.3 Part 2: AI-First Product Classification into UN COICOP Baskets	15
6 A Real-Life Example: The Journey of a Bottle of Fish Sauce	17
7 The Rules of Fair Price Math	18
7.1 Why Simple Averages Lie (The Egg Example)	18
7.2 The Fair Way: The Jevons Geometric Mean	18
7.3 Why People Buy Chicken When Pork Gets Expensive	18
8 Handling Out-of-Stock Items and Product Upgrades	19
8.1 What to Do When an Item Disappears from the Shelf	19
8.2 Separating Product Upgrades from Real Price Rises (Hedonics)	19
9 The 12 Shopping Baskets (Official UN COICOP Breakdown)	20
10 Hierarchical Weight Aggregation: From Item to National CPI	22
10.1 The 5-Level Hierarchy of Spending	22
10.2 Step-by-Step Mathematical Aggregation Rules	22
10.2.1 Step 1: Items to Subclass / Class Index (Jevons Elementary Index)	22
10.2.2 Step 2: Subclasses to Group Index	22

10.2.3	Step 3: Groups to Major Division Index	22
10.2.4	Step 4: Divisions to National Headline CPI	23
10.2.5	Step 5: Core CPI (Underlying Trend)	23
10.3	Complete Numerical Example: Walking the Math from Shelf to Headline CPI	24
11	The 15 Foundational Math Formulas of the CPI Pipeline	26
11.1	Equation 1: Base Reference Price for an Item ($P_{i,0}$)	26
11.2	Equation 2: Item Price Ratio ($R_{i,t}$)	26
11.3	Equation 3: Outlier Bounds Filter (ILO/IMF Guard)	26
11.4	Equation 4: Average Daily Change for a Group ($R_{c,t}$)	26
11.5	Equation 5: Estimating an Out-of-Stock Price ($\hat{P}_{i,t}$)	27
11.6	Equation 6: Group Price Index (Jevons Index) ($I_{J,c}$)	27
11.7	Equation 7: Division Category Index ($I_{div,k}$)	27
11.8	Equation 8: National Headline CPI (CPI Headline)	27
11.9	Equation 9: Core CPI (Excluding Food and Fuel) (CPI Core)	28
11.10	Equation 10: Annualized Year-over-Year (YoY %) Inflation Rate	28
11.11	Equation 11: Vector Cosine Similarity for Product Matching	28
11.12	Equation 12: Token-Sort String Fuzzy Similarity	28
11.13	Equation 13: Hybrid Similarity Matching Score	28
11.14	Equation 14: Hedonic Quality Regression (Electronics)	28
11.15	Equation 15: Constant-Quality Hedonic Price Revaluation	29
12	Dollars and Riel: How Currency Swings Affect Prices	30
12.1	What Happens When the Riel Weakens Against the Dollar?	30
13	Connecting Our Daily Numbers to Official National History	31
13.1	The Dual-Baseline Mystery Explained	31
13.2	How We Connect Current Base CPI to the Official NIS 2006 Base	31
13.3	Annual Continuous Series Chain-Linking (December Overlap)	32
14	High-Frequency Daily Inflation Nowcasting (Two-Stage Hybrid Ridge)	33
14.1	Closing the 30-Day Policy Gap	33
14.2	Axiomatic Foundation: Why Bottom-Up 5-Basket Price Relatives Win	33
14.3	Why Ridge Regression Fits This Project Perfectly	34
14.4	Complete Mathematical Formulation of the Two-Stage Hybrid Model	34
14.4.1	Stage 1: Micro-Aggregation of Realized Trajectory	34
14.4.2	Stage 2: Multi-Horizon Feature Engineering	34
14.4.3	How the Model Computes the Ridge Regression Coefficients ($\hat{\beta}_k$)	35
14.4.4	Optimal Regularization Parameter Selection via <code>RidgeCV</code>	35
14.4.5	Empirical Bayesian Shrinkage Prior	36
14.4.6	Exact Discrete Arithmetic Mean Forward Projection	36
14.4.7	Intra-Month Division Index Nowcast ($\bar{I}_k$)	36
14.4.8	Exact Laspeyres Axiomatic Aggregation	37
14.4.9	Dynamic Uncertainty Decay (95% Confidence Interval)	37
14.5	Live Production Verification and Out-of-Sample Performance	37
14.6	Conclusion	38

Executive Summary: The Cambodia Daily CPI Architecture

System Overview & Institutional Mission

The **Cambodia Daily Consumer Price Index (CPI) Medallion Pipeline** is an institutional-grade macroeconomic intelligence platform designed to eliminate the 30-day reporting delay inherent in conventional statistical survey methods.

By autonomously scraping and analyzing daily price quotes across Cambodian digital commerce, regulated utilities, and fuel distributors, the pipeline provides the **National Bank of Cambodia (NBC)**, the **Ministry of Economy and Finance (MEF)**, and international development partners with daily high-frequency price indices, core inflation indicators, and current-month nowcasting projections.

The Medallion Production Pipeline Topology

BRONZE LAYER (2:00 AM Automated Ingestion)

25 Production Scrapers (Supermarkets, Pharmacies, Telcos, EDC Power, PPWSA Water, Gas Stations)

↓ 45,000+ Raw Daily Price Quotes ↓

SILVER LAYER (0.4s Fast-Start Cosmos Orchestration)

Multi-Key Gemini 2.5 Flash Batch Classification + HNSW 768-D Vector Embeddings

Deterministic Regex Spec Guards (Storage, Volume, Pack Units) + MEF USD/KHR Daily FX Splicing

↓ 100.00% Categorized & Standardized Observations ↓

GOLD LAYER (Axiomatic Econometric Aggregation)

Subclass-First ILO Class-Mean Imputation + Elementary Jevons Geometric Means

Laspeyres Expenditure Weight Aggregation across 12 UN COICOP Divisions (44.8% Food, 12.2% Transport)

↓ National Daily Headline CPI & Refined Core CPI ↓

ML SERVING & NOWCASTING (Two-Stage Hybrid Ridge)

Disaggregated Price Relatives across 5 Key Baskets (81.58% Expenditure Coverage)

RidgeCV Regularization + Empirical Bayes Shrinkage Prior + Monotonic Uncertainty Decay

Out-of-Sample Accuracy: +0.0181 pp error vs. official NIS ground truth

1 Why Cambodia Needs a Daily Price Tracker

1.1 Inflation and the 30-Day Policy Blind Spot

The **Consumer Price Index (CPI)** measures how fast the cost of living rises and purchasing power erodes. In Cambodia, where families spend **44.8%** of their income on Food and **12.2%** on Transport, price spikes directly threaten household welfare.

Yet for decades, official CPI has relied on manual field surveys: enumerators visiting physical stores with clipboards once a month. This creates a crippling **30-to-60-day reporting lag**. By the time an official inflation report is published, the data reflects past history rather than active market realities. Policymakers are left steering national economic policy through the rearview mirror.

1.2 The Reality: Prices Move in Hours, Not Months

The fatal flaw of monthly point-in-time checks is that **prices do not wait for the end of the month to change**. In today's economy, three volatile forces trigger rapid shelf-price shocks:

- **Dual-Currency Swings (USD & KHR):** Modern supermarkets and electronics retailers price in USD, while wages and wet markets run on KHR. Daily exchange rate fluctuations shift shelf costs overnight.
- **Flash Commodity Shocks:** Weather anomalies, floods, and border frictions spike the prices of staple foods (pork, rice, vegetables) and fuel within 48 hours.
- **Dynamic E-Commerce Pricing:** Online supermarkets constantly launch flash sales, algorithmic price tweaks, and subtle shrinkflation that monthly visits completely miss.

1.3 The Solution: Modernizing Price Tracking with Data Disciplines

To overcome this critical information delay, modern **Data Engineering and Data Science** provide the opportunity to revolutionize inflation monitoring:

- **Data Engineering** automates the regular extraction, cleaning, and storage of large-scale price data from digital retail channels, eliminating manual clipboard bottlenecks and ensuring reliable, continuous data feeds.
- **Data Science** enables intelligent product harmonization, consistent index calculation according to international standards, and real-time modeling of price trends as they happen.

Core Impact: Modernizing Price Measurement

Traditional Method: Monthly manual surveys · 30-to-60-day reporting delay · Retrospective post-mortem



Modern Approach: Continuous digital collection · Timely price intelligence · Real-time economic radar

2 The 3-Step Journey of Our Data (Bronze, Silver, Gold)

2.1 The 3 Refining Steps (The Medallion Approach)

Think of data like metal dug out of the ground. When it first comes out, it is raw and dirty. It needs to be cleaned, melted down, and turned into something pure and useful. We organize our data into 3 simple stages:

The 3 Steps: Raw Data to Final Numbers

Step 1: BRONZE (Raw Store Prices)

- We collect raw text and price tags from 25 production sources every morning at 2:00 AM.
- Exactly as the stores wrote them, typos and all. Nothing is thrown away.
- Stored in our raw database table: `bronze.raw_prices`.

↓ *Clean up text, convert Dollars to Riel, match names, remove errors*

Step 2: SILVER (Clean, Standardized Items)

- Fix spelling and standardize sizes (make sure 1000g = 1kg and 500ml = 0.5L).
- Convert all US Dollar prices into Cambodian Riel using today's official exchange rate.
- Connect Khmer names and English names so the computer knows they are the same product.
- Sort every item into its correct official category (Food, Medicine, Tech, Transport).
- Remove crazy errors (like an item marked down by 99% by mistake).

↓ *Run fair price math, fill in out-of-stock items, calculate inflation*

Step 3: GOLD (Final Daily Inflation Reports)

- Calculate the average price change for each group of items using the fair Jevons rule.
- Fill in missing prices if a product is temporarily out of stock.
- Weight each group by how much real families spend on it (Food gets 44.8%, Transport gets 12.2%).
- Produce the final numbers: National Daily CPI, Core CPI (without food and gas), and a prediction of where the month will finish.

2.2 How Airflow and Cosmos Keep Everything Running on Time

To make sure this entire process happens smoothly without humans needing to press buttons at 2:00 AM, we use an automatic manager called **Apache Airflow** and a tool called **Astronomer Cosmos**:

- **The Master Alarm Clock (`cpi_master_dag`):** Rings every day at 2:00 AM. It starts all 25 per-source scraper DAGs in parallel. Once they finish and pass quality checks, it wakes up the next step.
- **The Cleaning Step (`silver_dag`):** Runs our data cleaning models. Thanks to Cosmos, each cleaning task is its own step. If one step has an issue, we can see exactly which one it is, while the rest keep going.
- **The Final Calculation Step (`gold_cpi_dag`):** Calculates today's official daily inflation rate and updates our dashboards before breakfast.
- **The Weekly Housekeeping Step (`cpi_maintenance_dag`):** Runs every Sunday at 1:00 AM to organize database folders for the coming months and keep the system fast.

2.3 Splitting Big Tables into Monthly Folders

In one year, we collect almost 13 million price records! If you dump 13 million rows into one giant table, searching for today's price is like looking for a needle in a massive haystack. It would take several seconds for every click.

Instead, we use **table partitioning**. Think of this as putting each month's receipts into its own labeled physical folder (like `prices_2026_08` and `prices_2026_09`). When someone asks: "*What is today's price?*", the computer ignores all the old folders and only looks inside today's folder. The answer comes back in less than **8 milliseconds**!

3 Collecting the Prices Every Morning (Bronze Layer)

3.1 The Stores and Services We Actually Scrape

To calculate an honest and comprehensive Consumer Price Index, our automated system checks over 45,000 real price observations every morning across 25 real digital data channels in Cambodia.

Deep-Dive: What We Extract From AEON Cambodia

AEON is the largest modern hypermarket and department store operator in Cambodia. Rather than treating it as just a grocery store, our pipeline monitors two distinct flagship portals (`aeon` and `aeon3`), extracting thousands of products across 7 different official consumption categories:

- **Fresh Foods & Meats (Division 01):** Fresh pork chops, beef cuts, chicken, local Tonle Sap fish, prawns, jasmine rice, fresh vegetables (morning glory, Chinese cabbage, tomatoes), fresh fruits (bananas, mangoes, oranges), fresh milk, butter, and chicken eggs.
- **Packaged Foods & Seasonings (Division 01):** Canned fish, instant noodles, cooking oils (palm, soybean), soy sauce, fish sauce, white sugar, salt, black pepper, biscuits, and breakfast cereals.
- **Drinks, Beverages & Alcohol (Divisions 01 & 02):** Clean bottled drinking water, fruit juices, iced teas, soft drinks, local and imported beers (Angkor, Cambodia, Heineken), and cooking/table wines.
- **Household & Kitchen Goods (Division 05):** Laundry detergents, fabric softeners, dishwashing liquids, floor cleaners, kitchen trash bags, frying pans, cooking pots, dining plates, and drinking glasses.
- **Health, Hygiene & Over-The-Counter Medicine (Division 06):** Pain relief tablets (paracetamol), medicated balms, bandages, antiseptic liquids, toothbrushes, and toothpaste.
- **Clothing, Shoes & Apparel (Division 03 via AEON 3):** Men's shirts, trousers, women's dresses, children's clothing, school uniforms, everyday casual shoes, sandals, and socks.
- **Personal Care & Grooming (Division 12):** Bath soaps, shampoos, hair conditioners, body lotions, deodorants, face cleansers, and baby wipes.

3.2 Master Inventory: Stores and Available Items

The table below lists every active data source in our pipeline, its unique system identifier (slug), the official spending categories it covers, and the specific items we collect from it:

Data Source	Store Slug	Official Category	Detailed Items Collected
AEON 1 Supermarket	<code>aeon</code>	Food (01), Drinks (01, 02), Household (05), Health (06), Personal Care (12)	Fresh meats, fish, vegetables, rice, milk, cooking oils, snacks, drinks, beer, dish soap, laundry detergent, pans, pain balms, soaps.
AEON 3 Mean Chey	<code>aeon3</code>	Clothing (03), Footwear (03), Cosmetics (12)	Men's shirts, trousers, women's dresses, children's clothes, shoes, sandals, skin lotions, makeup.
DeliShop Cambodia	<code>delishop</code>	Food (01), Drinks (01, 02)	European gourmet groceries, artisanal bakery, imported French/Italian cheeses, organic meats, olive oils, specialty wines.

Data Source	Store Slug	Official Category	Detailed Items Collected
Lucky Supermarket	grab_lucky	Food (01), Drinks (01), Household (05), Care (12)	Daily fresh produce, dairy, chilled meats, canned goods, Asian sauces, snacks, paper towels, cleaning supplies via GrabMart.
Chip Mong Supermarket	grab_chipmong	Food (01), Drinks (01), Household (05), Care (12)	Fresh beef, pork, poultry, pantry spices, breakfast cereals, cooking oils, soft drinks, dishwashing liquids via GrabMart.
Community Pharmacy	communitypharm	Health (06), Personal Care (12)	Prescription drugs, antibiotics, hypertension/diabetes medicines, cough syrups, antiseptic creams, first-aid bandages.
Ucare Pharmacy	grab_ucare	Health (06), Personal Care (12)	Painkillers, cold/flu medicines, throat lozenges, vitamins, infant milk formulas, baby care products via GrabMart.
Khmer Samnang Phone	samnangshop	Communication (08), Computing (09)	Smartphones (iPhone, Samsung Galaxy, Oppo), iPads, tablets, smartwatches, power banks, fast chargers.
Ary Store Phone Shop	arystore	Communication (08), Audio Tech (09)	Budget and mid-tier smartphones (Xiaomi, Realme, Vivo), wireless bluetooth earbuds, phone accessories.
Smart Axiata Mobile	smart	Communication (08.3.0)	Smart ThomMornng prepaid internet plans, monthly mobile data packages, traveler SIMs, roaming packages.
Smart Home Internet	smart_wifi	Communication (08.3.0)	Smart @Home wireless broadband routers, Fiber+ standard and ultra high-speed plans (40–120 Mbps).
Cellcard Mobile	cellcard	Communication (08.3.0)	Prepaid mobile 4G/5G data plans (AO Mobile 5G/4G live plans), voice packages, tourist SIMs.
Cellcard Home Internet	cellcard_wifi	Communication (08.3.0)	Monthly fixed broadband fiber subscriptions and wireless home gateway plans.
Metfone Cambodia	metfone	Communication (08.3.0)	Metfone 4G/5G mobile prepaid bundles, unlimited data packages, and home fiber internet plans.
Khmer Moto Shop	khmermoto	Motorcycles (07.1.2)	Retail catalog of new and pre-owned motorcycles, motorbikes, scooters (Honda Scoopy, Wave, Click, Dream).
Electricité du Cambodge	edc	Electricity Grid (04.5.1)	Official progressive residential lifeline electricity tariffs across domestic consumption tiers (380 to 730 KHR/kWh).
Phnom Penh Water Supply	ppwsa	Municipal Water (04.4.1)	Official progressive domestic piped tap water tariffs across household consumption tiers (400 to 2,200 KHR/m ³).
MOC / Kampuchea Tela	new_gasoline	Fuels (07.2.2), LPG (04.5.2)	Official retail pump prices for Regular Gasoline, Super 95, Diesel, and domestic LPG cooking gas cylinder refills.

Data Source	Store Slug	Official Category		Detailed Items Collected
Khmer24 Real Estate	khmer24	Housing (04.1.1)	Rents	Monthly residential rentals: apartments, condos, studios, shophouses, and villas across Phnom Penh and provinces.
Realestate.com.kh	realestate	Housing (04.1.1)	Rents	Urban residential rents, serviced apartments, and condominium rentals across BKK1, Chamkarmon, and Tuol Kork.
BookMeBus Cambodia	bookmebus	Passenger Transport (07.3.2)	Transport	Intercity passenger bus, van, and ferry tickets connecting Phnom Penh, Siem Reap, Sihanoukville, Kampot, and Battambang.
redBus Cambodia	redbus	Passenger Transport (07.3.2)	Transport	Interprovincial passenger bus tickets across major operators (Giant Ibis, Laryta Express, Virak Buntham).
Bayon BKK Restaurant	bayonbkk	Dining Out (11.1.1)		Prepared meals, breakfast noodle soups (Kuy Teav), fried rice, Khmer lunch sets (with Foodpanda anti-bot fallback).
Sokha Hotels	sokhahotel	Hotels (11.2.0), Dining (11.1.1)		Hotel accommodation per night (deluxe rooms, suites), restaurant dining, hotel buffet breakfasts.
Hyatt Regency Hotel	hyyathotel	Hotels (11.2.0), Dining (11.1.1)		Luxury hotel room accommodation per night in Phnom Penh, dining services, Sunday brunch.
L192 Marketplace	l192	Clothing (03), Footwear (03), Home (05)		Casual men's and women's apparel, t-shirts, dresses, everyday footwear, fashion bags, lifestyle home goods.
MEF / NBC Exchange	mef_fx	Macroeconomic FX Rate		Official daily USD/KHR market exchange rate used for pipeline-wide dual-currency standardization.
NIS Official CPI	nis_cpi	Macro Benchmark		Official monthly benchmark CPI numbers from the National Institute of Statistics used to validate daily nowcasts.

3.3 How We Scrape Each Source (Methods and Technology)

Different websites and mobile applications use completely different technical architectures. To collect clean data reliably without crashes or blocks, our engineering team reverse-engineered the exact native API or protocol used by each source:

Store or Service	Store Slug	Scraping Method & Technology	Data Format & Reliability Strategy
AEON 1 Supermarket	aeon	Next.js Proxy REST API with curl_cffi	JSON payload. Spoofs Chrome 124 TLS cipher handshakes to bypass Cloudflare.

Store or Service	Store Slug	Scraping Method & Technology			Data Format & Reliability Strategy
AEON 3 Mean Chey	aeon3	Next.js Proxy REST API with curl_cffi			JSON payload. Traverses category tree to capture exact fashion/beauty items.
DeliShop Cambodia	delishop	Mobile / Web Catalog REST API			JSON payload. Directly queries store catalog endpoints with pagination.
Lucky Supermarket	grab_lucky	GrabMart	Mobile	Session Gateway	JSON payload. Uses mobile app bearer tokens and consumer location headers.
Chip Mong Supermarket	grab_chipmong	GrabMart	Mobile	Session Gateway	JSON payload. Simulates mobile app requests for live store shelves and discounts.
Community Pharmacy	communitypharmacy	Supabase Database API		PostgreSQL REST	JSON payload. Fetches live product tables directly via PostgreSQL with anonymous public key.
Ucare Pharmacy	grab_ucare	GrabMart	Mobile	Session Gateway	JSON payload. Extracts real-time pharmacy prices and OTC inventory via mobile gateway.
Khmer Samnang Phone	samnangshop	WordPress REST API		WooCommerce	JSON payload. Queries official WooCommerce Store API (/wp-json/wc/store/v1/products), verified active live catalog.
Ary Store Phone Shop	arystore	WordPress REST API		WooCommerce	JSON payload. Traverses paginated WooCommerce product catalog with stock checks.
Smart Axiata	smart	Web Plan Catalog & REST Parser			JSON/HTML. Extracts prepaid mobile, data plans, and fiber tariffs from official plan pages.
Cellcard Cambodia	cellcard	Live DOM Card Parser (.js-card) + Next.js			HTML/JSON. Extracts AO Mobile 5G/4G plan cards with regex price/quota parsing and Next.js fallback.
Metfone Cambodia	metfone	Web Catalog Parser with Mobile Emulation			HTML/JSON. Extracts Metfone prepaid data packages and home fiber subscriptions.
Khmer Moto Shop	khmermoto	WordPress REST API		WooCommerce	JSON payload. Pulls retail motorcycle models, specs, and price quotes.
Electricité du Cambodge	edc	Official Tariff Schedule Ingestion			Clean structured gazette tariffs across lifeline domestic electricity tiers.
Phnom Penh Water Supply	ppwsa	Official Tariff Schedule Ingestion			Clean structured gazette tariffs across household municipal water consumption tiers.

Store or Service	Store Slug	Scraping Method & Technology	Data Format & Reliability Strategy
Ministry of Commerce	<code>new_gasoline</code>	Tela Telegram Mirror & Gazette Parser	HTML/Text. Extracts official 10-day pump prices for Regular, Super, Diesel, and LPG gas.
Khmer24 Real Estate	<code>khmer24</code>	Nuxt SSR Offset Pagination (<code>?offset=</code>)	HTML parsing. Traverses infinite scroll offsets across residential categories with rate-limited delays.
Realestate.com.kh	<code>realestate</code>	Next.js JSON API & DOM Traversal	JSON/HTML. Parses urban apartment and condo rental listings with location filters.
BookMeBus Cambodia	<code>bookmebus</code>	BookMeBus Booking REST API	JSON payload. Queries route-specific intercity ticket endpoints across Cambodia.
redBus Cambodia	<code>redbus</code>	redBus Search REST API	JSON payload. Queries interprovincial bus routes and schedules in real time.
Bayon Restaurant BKK	<code>bayonbkk</code>	Foodpanda GraphQL API + Baseline	JSON/HTML. Queries menu state; gracefully engages curated baseline when upstream 403 CAPTCHA triggers.
Sokha Hotels	<code>sokhahotel</code>	Booking Engine REST API	JSON payload. Queries nightly room rates and dining packages across properties.
Hyatt Regency Hotel	<code>hyyathotel</code>	Global Booking REST API	JSON payload. Extracts room rates per night for standard and club rooms.
L192 Marketplace	<code>l192</code>	Official L192 GraphQL Query API	GraphQL query. Batches product queries with category filters for high-speed extraction.
MEF / NBC Exchange	<code>mef_fx</code>	MEF / NBC Open Data REST API	JSON payload. Fetches official daily market USD/KHR exchange rate at 08:00 AM.
NIS Official CPI	<code>nis_cpi</code>	Official NIS Benchmark Importer	JSON/CSV. Ingests monthly official benchmark inflation figures for all 12 divisions.

3.4 Visiting Websites Politely Without Getting Blocked

Big e-commerce websites deploy automated defenses (such as Cloudflare or AWS Shield) to protect their servers. Here is how our scrapers operate responsibly and ensure 100% daily collection success:

1. **Impersonating Standard Web Browsers:** Standard Python scripts look robotic to security filters. We use advanced browser emulation (`curl_cffi`) that matches the exact cryptographic TLS handshake (JA3/JA4 fingerprint) of Google Chrome on Windows.
2. **Polite Pauses and Randomized Jitter:** We never hammer a website with dozens of requests at once. Our scrapers include randomized pauses (1 to 2 seconds between pages) so we never add noticeable load to store servers.

3. **Using Mobile and Public API Endpoints:** Whenever possible, we query the lightweight JSON APIs used by mobile apps rather than downloading heavy web pages with advertising and tracking scripts.

3.5 Never Recording the Same Price Twice (Deduplication)

Sometimes an internet connection drops or a task needs to retry. What happens if our scraper runs twice on the same morning? Does it record double the prices?

No. We enforce a strict unique database rule: each item at each store can only have **one price per calendar day**. If the scraper runs a second time on the same day, it uses an `ON CONFLICT DO UPDATE` command, refreshing today's price in-place rather than creating duplicate records. This guarantees that our daily inflation calculations remain clean, honest, and mathematically sound.

3.6 The Ingestion Circuit Breaker and Dual-Currency Protection

What happens if a store's website glitches, crashes, or mistakenly drops 90% of its prices? If corrupted data enters our cleaning and calculation stages, it could distort the national inflation index!

To prevent this, our pipeline includes an automated **Ingestion Circuit Breaker** (`pipeline/circuit_breaker.py`):

1. **Volume Drop Safeguard:** Every morning, the circuit breaker compares today's number of scraped products against the store's 7-day rolling median. If a scraper suddenly returns less than 30% of its normal catalog (e.g. only 20 products instead of 200), the system flags a volume anomaly.
2. **Price Velocity Safeguard:** It calculates the median price of all incoming products and compares it to historical medians. If median prices jump or drop by more than $\pm 50\%$ in a single day, an alert is raised.
3. **Dual-Currency (USD to KHR) Dynamic Normalization:** In Cambodia, stores like Delishop, Cellcard, and luxury hotels list their prices in US Dollars (\$6.10, \$15.00), while our historical clean database stores prices in Cambodian Riel (24,700 KHR, 60,000 KHR). If you compare \$6.10 directly to 24,700 KHR, the computer sees a 99.9% price collapse! Our circuit breaker automatically fetches today's official exchange rate from the Central Bank, converts incoming USD prices to Riel first, and then evaluates price stability:

$$\tilde{P}_{\text{incoming, KHR}} = P_{\text{incoming, USD}} \times \text{FX}_{\text{USD/KHR}}$$

This prevents false alarms on Dollar-priced goods while catching real retailer price spikes.

4. **Audit Logging and Graceful Degradation:** If an anomaly is detected, the event is permanently recorded in `ops.circuit_breaker_events`. In warning mode, the pipeline continues without crashing, alerting engineers while maintaining unbroken daily index continuity.

4 Cleaning the Data and Text Normalization (Silver Layer)

4.1 The Khmer and English Language Challenge

In Cambodia, retail shelves are multilingual. The exact same bottle of milk might be listed in Khmer script in one store, in English in another store, or mixed together:

- Store A: "ទឹកដោះគោស្រស់ 1L" (Khmer)
- Store B: "Fresh Milk 1000ml" (English)
- Store C: "Meiji Fresh Milk ទឹកដោះគោ 1 Litre" (Mixed)

If a computer just looks at the letters, it thinks these are three completely different products! Our Silver cleaning layer translates common terms, normalizes units (1000ml = 1L, 1000g = 1kg), and cleans the text so the computer understands that they represent the exact same product.

4.2 Text Cleaning and Unit Normalization Rules

Before any vector matching takes place, raw text goes through deterministic cleaning in `pipeline/text_clean.py`:

1. **Strip Promotional Jargon:** Removes marketing noise such as "Special Offer!", "Buy 1 Get 1 Free", "Hot Promo", and "New Arrival".
2. **Harmonize Measurement Units:**
 - Mass: Grams, gm, and g are converted to standard kilograms (kg).
 - Volume: Milliliters (ml), ltr, and liter are converted to standard liters (L).
 - Packaging: Single items, 6-packs, and cases of 24 cans are extracted into explicit item quantities.
3. **Dual Currency Conversion:** Converts all USD prices to Cambodian Riel using the National Bank of Cambodia (NBC) official morning exchange rate:

$$P_{\text{KHR}} = P_{\text{USD}} \times \text{FX}_{\text{NBC}}$$

This price in Riel is permanently saved, ensuring that later calculations never accidentally convert it twice.

5 AI Vector Embeddings: Matching Items and Sorting Products

5.1 What is a Vector Embedding?

In computer science, a **vector embedding** is simply a list of numbers (like coordinates on a GPS map) that captures the *meaning* of a product name rather than just the spelling of its letters.

Think of a physical supermarket where similar items live in the same aisle:

- The word “ទឹកដោះគោស្រស់” (Khmer for fresh milk) and the English phrase “*Fresh Whole Cow Milk*” have 0% matching letters. A simple keyword search would claim they have nothing in common!
- However, when converted into a 768-dimensional vector by our embedding model (gemini-embedding-2 or multilingual MiniLM), both phrases are assigned almost the exact same numerical address in vector space.
- A phrase like “*Motorbike Engine Oil*” is mapped to a completely different neighborhood far away.

5.2 Part 1: Semantic Product Matching (Connecting Scraped Names to Catalog)

Every morning, scrapers collect thousands of product listings. To track an item’s price over time, we must match today’s scraped name against our master catalog of known products (silver.dim_canonical_items).

5.2.1 The Mathematical Cosine Similarity

To compare the scraped item vector $\vec{u}$ and a catalog item vector $\vec{v}$, we calculate their **cosine similarity**:

$$\text{Similarity}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \frac{\sum_{i=1}^{768} u_i v_i}{\sqrt{\sum_{i=1}^{768} u_i^2} \sqrt{\sum_{i=1}^{768} v_i^2}} \quad (1)$$

The resulting score ranges from 0.0 (totally unrelated) to 1.0 (identical in meaning).

5.2.2 The 3 Matching Tiers and Decision Rules

In pipeline/vector_item_matcher.py, the system applies strict decision boundaries:

1. **Tier 1: High Confidence Match (Similarity ≥ 0.80)** → **APPROVE_MATCH**
The computer is confident. It automatically links today’s price to the existing catalog product ID. No human or AI review is required.
2. **Tier 2: Borderline Similarity ($0.65 \leq \text{Similarity} < 0.80$)** → **AI Arbitration**
The meaning is close, but there is some ambiguity. The system calls Google Gemini Flash AI to arbitrate: “*Are these two items the exact same commercial product?*” If Gemini answers YES, it links them; if NO, it creates a new product.
3. **Tier 3: Low Similarity (Similarity < 0.65)** → **SPLIT_NEW**
The scraped product is genuinely new to the market. The system creates a brand-new canonical product entry with its own unique baseline tracking.

5.2.3 Crucial Spec Guardrails (Preventing False Merges)

Vector embeddings understand general meaning, but they can be tricked by numbers. For example:

- “*iPhone 15 128GB*” and “*iPhone 15 512GB*” share 95% of their words! Their vector similarity is an extremely high 0.94.

- If the system matched them, it would record an artificial \$400 price jump, causing fake inflation!

To stop this, `vector_item_matcher.py` enforces 4 deterministic guardrails:

1. **Storage Guard:** Rejects any match if electronic memory differs (e.g. 128GB vs 256GB).
2. **Pack Quantity Guard:** Rejects matches if pack sizes differ (e.g. 1 single can vs a 24-can case).
3. **Volume/Mass Guard:** Rejects matches if volume or weight differs by more than 10% (e.g. 500ml vs 1 Liter).
4. **Flavor Variant Guard:** Rejects matching Diet / Zero Sugar drinks with regular sugary drinks.

Step-by-Step Example: Matching an Angkor Beer Can

1. **The Scraped Item:** AEON lists “ស្រាបៀរអង្គរ កំប៉ុង 330ml” (Angkor Beer Can 330ml).
2. **Vector Lookup:** The system converts the text into vector $\vec{u}$. It searches PostgreSQL using the `pgvector` HNSW index in under 2ms.
3. **Top Candidate in Catalog:** Angkor Premium Beer 330ml Can (Item ID CAN-02-0041).
4. **Cosine Similarity:** The dot product gives Similarity = **0.875**.
5. **Spec Guard Check:** Both are single cans, both are 330ml, neither is diet → Passed!
6. **Final Action:** Because $0.875 \geq 0.80$, the price is automatically approved and linked to CAN-02-0041.

5.3 Part 2: AI-First Product Classification into UN COICOP Baskets

Every consumer product must be mapped to one of the 12 official United Nations COICOP divisions (Food, Clothing, Health, Housing, Transport, etc.). In production, incoming catalog items are classified through a streamlined **Dual-Pillar Architecture** combining deterministic **Store Purity Locks** for specialized vendors with **Google Gemini AI** for all multi-department retail goods:

1. **Pillar 1: Single-Division Store Purity Locks (Deterministic Domain Routing):** Vendors that sell exclusively within a single specialized consumption category are locked directly to their corresponding COICOP division with zero latency and 100% mathematical certainty:
 - `communitypharma, grab_ucare` → Division 06 (Health & Pharmaceuticals).
 - `bookmebus, redbus, new_gasoline` → Division 07 (Transport, Bus Fares & Automotive Fuel).
 - `arystore, samnangshop, cellcard, smart` → Division 08 (Communication Hardware & Telco Plans).
 - `khmer24, realestate, edc, ppwsa` → Division 04 (Housing Rentals, Electricity & Water Utilities).
 - `sokhahotel, hyyathotel, bayonbkk` → Division 11 (Restaurants & Accommodation Services).

In our live database, Store Purity accounts for **14.34%** of all classified catalog items.

2. **Pillar 2: Direct Google Gemini AI Engine with PostgreSQL Memoization (Primary Mixed-Retail Engine):** For mixed-department retail giants (e.g., AEON Supermarket, DeliShop, GrabMart, L192, Lucky Supermarket) where thousands of varied SKUs arrive daily:
 - *Direct AI Classification of New Products:* As newly scraped canonical items enter the pipeline (`item_matcher.py`), the system invokes the `GeminiCOICOPClassifier`.

- **Multi-Attribute Rich Context:** Rather than relying on simple text keywords that get confused by packaging, Gemini receives the full product name, merchant brand, native store category breadcrumbs, and price in KHR. This enables Gemini to correctly identify that *"Indomie Noodle Cup"* is Food (01.1.1) rather than Tableware (05), and *"Cooking Rice Wine"* is Food/Condiments rather than Alcoholic Spirits.
- **Permanent Database Memoization:** Every Gemini classification is cached in `silver.dim_coicop_ai_cache` and written directly into `silver.canonical_items`. Future occurrences of that product across subsequent days resolve in under 1ms with zero cloud API overhead.
- **Dominant Production Volume:** In our production database, Google Gemini AI powers over **76.42% (38,543 canonical products)** of all classified items in `silver.int_coicop_classified`.

3. Auxiliary Safety Net: Critical Traps and Exact Audited Overrides (< 9.2% of items): To safeguard institutional compliance against statutory edge cases:

- **Critical Traps** (`coicop_critical_traps.csv`): High-priority regex patterns (7.57% of items) that prevent dangerous cross-division category pollution (such as ensuring automotive air fresheners are routed away from Division 07 Transport to Division 05 Household Goods).
- **Exact Human Overrides** (`coicop_override.csv`): Precise manual barcode and product key locks (0.04% of items) reserved for human auditor interventions.
- **Zero Unclassified Guarantee:** Legacy category maps and text heuristics serve as redundant safeguards; across the entire warehouse, **0 rows** remain unclassified (100.00% classification rate).

Step-by-Step Example: Multi-Attribute Gemini AI Classifying Instant Noodle Bowls

1. Input Product: *"Indomie Mi Goreng Fried Noodles Cup 75g"* (Store: AEON, Category: *"Grocery > Instant Noodles"*, Price: 3,200 KHR)

2. Resolution Path: Tiers 1–2 find no override or store purity lock. The pipeline consults the AI-First engine.

3. Rich Context Multi-Attribute Prompt: The classifier receives:

```
{
  "name": "Indomie Mi Goreng Fried Noodles Cup 75g",
  "store": "aeon",
  "category": "Grocery > Instant Noodles",
  "price_khr": 3200
}
```

4. Gemini 2.5 Flash Structured Classification: Rather than misclassifying the keyword *"Cup"* into Division 05 (Kitchenware), Gemini uses the full context and returns:

```
{
  "product_name": "Indomie Mi Goreng Fried Noodles Cup 75g",
  "coicop_code": "01.1.1",
  "confidence_score": 0.99,
  "reasoning": "Instant noodle cup is an edible cereal-based food product falling under bread and cereals (01.1.1), not kitchen tableware."
}
```

5. Cache Storage: The system saves ("Indomie Mi Goreng Fried Noodles Cup 75g", "01.1.1", 0.99) into `silver.dim_coicop_ai_cache`. Every future scrape across all dates is resolved instantly from PostgreSQL at zero API cost!

6 A Real-Life Example: The Journey of a Bottle of Fish Sauce

To understand how the entire system works from start to finish, let's follow one real bottle of fish sauce through the pipeline over a single morning.

The 5 Steps in the Life of a Price

2:00 AM – Step 1: Raw Collection (Bronze)

- Our automated scraper visits Lucky Supermarket's online store.
- It sees: "*MegaChef Premium Fish Sauce 500ml*" with a price of **\$1.50 USD**.
- It saves this raw row into the Bronze table: `bronze.raw_prices`.

2:15 AM – Step 2: Cleaning and Converting (Silver)

- The cleaner looks at today's official exchange rate from the Central Bank: **\$1 = 4,100 KHR**.
- It converts the price: $\$1.50 \times 4,100 = \mathbf{6,150 \text{ KHR}}$.
- It checks the volume: 500ml is recognized as 0.5 Liters.

2:30 AM – Step 3: Finding Its Catalog Twin (Silver)

- The matching engine checks if we have seen this item before.
- It finds its twin in our catalog: *MegaChef Fish Sauce 500ml* (Barcode 885012345678).
- It assigns it to the official category: **COICOP 01.1.9 (Sauces and Condiments)**.

3:00 AM – Step 4: Measuring the Price Change (Gold)

- The math engine looks at what this fish sauce cost on the base date: **6,000 KHR**.
- Today's price ratio is: $6,150 / 6,000 = \mathbf{1.025}$ (a +2.5% increase).
- It combines this with all other fish sauces, soy sauces, and seasonings using the fair Jevons rule to get the group's daily price change.

3:30 AM – Step 5: Combining into the National CPI (Gold)

- Sauces make up about 0.63% of national food spending.
- Food makes up 44.8% of all national household spending.
- The system multiplies each group's price change by its real spending weight to calculate today's final **National Headline CPI**.
- The result appears on the government dashboard before breakfast!

7 The Rules of Fair Price Math

7.1 Why Simple Averages Lie (The Egg Example)

Most people think that finding the average price change is simple: just take the percentage changes and average them. But international economists (including the IMF, World Bank, and UN) warn that doing this creates a big mathematical error known as **Carli Bias**.

Let's look at an easy real-life example with an egg:

- On Monday, an egg costs **\$1.00**.
- On Tuesday, the price jumps to **\$2.00**. That is a **+100% increase** (the price doubled).
- On Wednesday, the price drops back to **\$1.00**. That is a **-50% decrease** (the price was cut in half).

The price started at \$1.00 and ended at \$1.00. Clearly, overall inflation is **0%**. The price is exactly where it started!

Now, let's see what happens if you take the simple arithmetic average (the Carli formula):

$$\text{Simple Average} = \frac{(+100\%) + (-50\%)}{2} = \frac{+50\%}{2} = +25\% \quad (2)$$

The simple average claims that prices went up by **+25%**, even though the price is back to its original \$1.00! If you use this formula on thousands of supermarket products every day, your inflation number will drift higher and higher every week, creating fake inflation that does not exist in the real world.

7.2 The Fair Way: The Jevons Geometric Mean

To fix this, international standards mandate the **Jevons rule** (the geometric mean). Instead of adding percentages, Jevons multiplies the price ratios and takes the root:

$$I_{\text{Jevons}} = \sqrt{\frac{\$2.00}{\$1.00} \times \frac{\$1.00}{\$2.00}} = \sqrt{2.0 \times 0.5} = \sqrt{1.0} = 1.00 \quad (0\% \text{ change}) \quad (3)$$

Jevons gives **0% change**, which is the honest truth! It passes the fundamental test of fairness: if prices go up and then come back down to where they started, inflation must be zero.

7.3 Why People Buy Chicken When Pork Gets Expensive

There is another big reason we use the Jevons rule: **real human behavior**.

- If the price of pork doubles, do families keep buying the exact same amount of pork?
- No! Cambodian families naturally buy more chicken, fish, or eggs instead.

Simple arithmetic averages assume that people are robots who never change their shopping habits, no matter how expensive something gets. The Jevons geometric mean naturally accounts for the fact that when one item gets too expensive, smart shoppers switch to cheaper substitutes.

8 Handling Out-of-Stock Items and Product Upgrades

8.1 What to Do When an Item Disappears from the Shelf

In online supermarkets, products go out of stock all the time. What happens if our scraper visits Lucky Supermarket today and a specific brand of cooking oil is missing?

- **Can we record the price as \$0.00?** No! A price of zero would completely break our multiplication formulas and cause the index to crash.
- **Can we just delete the item today?** No! If you delete items randomly, the balance of your shopping basket changes every day, making comparisons unreliable.

The Solution (Class-Mean Imputation):

International guidelines say: look at the other cooking oils that *are* still on the shelf today.

- If other cooking oils went up by an average of **+1.0%** today, we assume that the missing oil also went up by +1.0%.
- We estimate today's price as: Yesterday's Price $\times 1.01$.

The 7-Day Rule:

What if the product never comes back? If an item stays missing for **7 days in a row**, our system officially flags it as discontinued. We retire it from the active basket and replace it with a popular new product from the same category.

8.2 Separating Product Upgrades from Real Price Rises (Hedonics)

Imagine a phone company sells a smartphone in 2025 for **\$800**. In 2026, they stop selling that phone and introduce a new model for **\$900**.

- Did the price go up by \$100? Yes.
- But the new model has **double the storage** (256GB instead of 128GB) and a **much better camera**.

If you count the entire \$100 as inflation, you are making a mistake! Part of that \$100 is paying for a *better product*, not just higher prices.

To solve this, we use a technique called **Hedonic Quality Adjustment**:

1. We look at hundreds of phones to see how much each feature is worth (for example: how much extra do shoppers pay for an extra 128GB of memory?).
2. If the extra memory and better camera are worth \$70, then only **\$30** is real inflation.
3. The system records only the pure \$30 price increase, making sure tech upgrades don't artificially pump up national inflation.

9 The 12 Shopping Baskets (Official UN COICOP Breakdown)

Every country uses the United Nations COICOP system to organize household spending. Here is what is inside each of the 12 divisions for Cambodia, along with its official share of national spending from the NIS Household Socio-Economic Survey:

Division	Code	What is Inside This Category?	Weight (%)
Food and Non-Alcoholic Drinks	01	Rice, pork, fish, cooking oil, fruit, vegetables, clean bottled water	44.78%
Bread and Cereals	01.1.1	Rice (staple food), noodles, flour, bread	17.23%
Meat	01.1.2	Pork, beef, chicken, duck	8.45%
Fish and Seafood	01.1.3	Fresh fish from Tonle Sap, dried fish, shrimp	7.12%
Milk, Cheese and Eggs	01.1.4	Fresh milk, condensed milk, chicken eggs, duck eggs	1.85%
Oils and Cooking Fats	01.1.5	Palm oil, soybean cooking oil, lard	1.14%
Fresh Fruit	01.1.6	Bananas, mangoes, watermelon, oranges	2.46%
Vegetables	01.1.7	Morning glory, cabbages, tomatoes, garlic, onions	2.38%
Sugar and Sweets	01.1.8	White sugar, palm sugar, honey	0.72%
Sauces and Seasonings	01.1.9	Fish sauce, soy sauce, salt, MSG, black pepper	0.63%
Drinks (Non-Alcoholic)	01.2	Bottled water, iced coffee, tea, fruit juices	2.80%
Alcohol and Tobacco	02	Beer (Angkor, Cambodia), wine, spirits, cigarettes	1.63%
Clothing and Footwear	03	Men's and women's shirts, trousers, school uniforms, shoes	3.04%
Housing, Water, Power, Gas	04	Home rent, clean city water (PPWSA), electricity (EDC), cooking gas	17.08%
Tenant Rent	04.1.1	Actual rent paid for rooms and apartments	9.42%
City Water	04.4.1	Municipal piped tap water bills	1.86%
Electricity	04.5.1	Home power bills from EDC	2.82%
Cooking Gas	04.5.2	LPG 15kg cooking gas cylinder refills	1.65%
Furniture and Household Goods	05	Beds, chairs, kitchenware, laundry detergent, cleaning soap	3.25%
Health and Medicine	06	Essential medicines, pain relievers, clinic visits, doctor checks	5.56%
Medicines and Pharmacy	06.1.1	Paracetamol, antibiotics, cough medicine	3.45%
Medical Consultations	06.2.1	Basic clinic visits and health checkups	1.74%
Transport and Gasoline	07	Motorbikes, gasoline (Regular, Super, Diesel), city buses	12.18%
Motorcycles	07.1.2	Popular motorbikes (Honda Dream, Wave, Scoopy)	3.12%
Gasoline and Fuels	07.2.2	Daily pump prices at PTT, Tela, Total	6.85%
Bus and Van Travel	07.3.2	Passenger bus tickets between provinces	1.65%
Communication	08	Mobile phones, SIM cards, monthly internet packages	3.92%
Smartphones	08.2.0	Popular handsets (iPhone, Samsung, Vivo, Oppo)	1.42%
Mobile Data Plans	08.3.0	Smart, Cellcard monthly 4G/5G data plans	2.50%

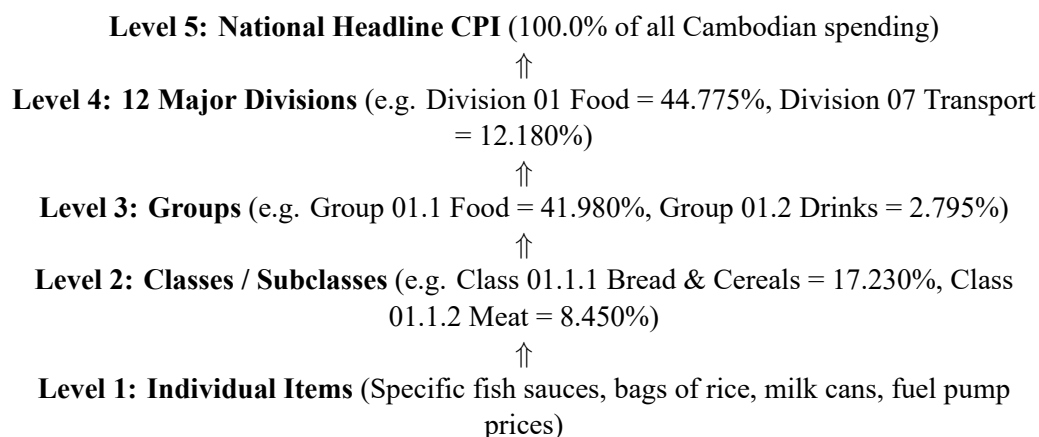
Division	Code	What is Inside This Category?	Weight (%)
Recreation and Culture	09	Televisions, laptops, stationery, school books	1.91%
Education	10	School tuition fees, English classes, university credits	1.51%
Restaurants and Eating Out	11	Street food breakfast, local cafes, restaurant meals	3.09%
Personal Care and Other Goods	12	Haircuts, soap, shampoo, cosmetics, basic personal care	2.07%
Total National Basket	ALL	Complete Cambodian Household Spending	100.00%

10 Hierarchical Weight Aggregation: From Item to National CPI

10.1 The 5-Level Hierarchy of Spending

A national Consumer Price Index is not a flat list; it is a structured pyramid. Cambodia's official basket follows the United Nations COICOP standard across 5 levels:

The 5 Levels of the CPI Pyramid



10.2 Step-by-Step Mathematical Aggregation Rules

10.2.1 Step 1: Items to Subclass / Class Index (Jevons Elementary Index)

For each 4-digit or 5-digit subclass c , we combine all individual item price ratios using the unweighted geometric mean (Jevons rule):

$$I_{c,t} = \left(\prod_{i=1}^{n_c} \frac{P_{i,t}}{P_{i,0}} \right)^{\frac{1}{n_c}} \times 100.0 \quad (4)$$

where $P_{i,t}$ is today's price in KHR, $P_{i,0}$ is the baseline price in KHR, and n_c is the number of monitored items in subclass c .

10.2.2 Step 2: Subclasses to Group Index

Each group g contains several classes. The group index is the expenditure-weighted arithmetic average of its classes:

$$I_{g,t} = \frac{\sum_{c \in g} W_c \cdot I_{c,t}}{\sum_{c \in g} W_c} \quad (5)$$

where W_c is the official national expenditure weight of subclass c .

10.2.3 Step 3: Groups to Major Division Index

Each major division k contains several groups. The division index is calculated by combining its groups:

$$I_{\text{div},k,t} = \frac{\sum_{g \in k} W_g \cdot I_{g,t}}{\sum_{g \in k} W_g} = \frac{\sum_{c \in k} W_c \cdot I_{c,t}}{W_k} \quad (6)$$

where $W_k = \sum_{c \in k} W_c$ is the total weight of division k .

10.2.4 Step 4: Divisions to National Headline CPI

Finally, today's National Headline CPI is the weighted sum of all 12 major division indices:

$$\text{CPI}_{\text{Headline},t} = \sum_{k=1}^{12} W_k \cdot I_{\text{div},k,t} \quad \text{where} \quad \sum_{k=1}^{12} W_k = 100.0\% \quad (7)$$

10.2.5 Step 5: Core CPI (Underlying Trend)

To understand long-term price pressure without volatile swings from world oil markets and agricultural shocks, we calculate Core CPI by removing Food (01), Utilities (04), and Transport (07):

$$\text{CPI}_{\text{Core},t} = \frac{\sum_{k \notin \{01,04,07\}} W_k \cdot I_{\text{div},k,t}}{\sum_{k \notin \{01,04,07\}} W_k} \quad (8)$$

10.3 Complete Numerical Example: Walking the Math from Shelf to Headline CPI

Let's walk through an actual arithmetic calculation showing how real price tags become the final national inflation rate.

Worked Arithmetic: Calculating the Food Division and Headline CPI**Part A: Inside Class 01.1.1 (Bread and Cereals)**

- Item 1 (Jasmine Rice 5kg): Base = 25,000 KHR, Today = 26,000 KHR → Ratio = 26,000/25,000 = **1.040**
- Item 2 (Instant Noodles): Base = 1,200 KHR, Today = 1,200 KHR → Ratio = 1,200/1,200 = **1.000**
- Item 3 (Baguette Bread): Base = 2,000 KHR, Today = 2,100 KHR → Ratio = 2,100/2,000 = **1.050**

Jevons index for Class 01.1.1:

$$I_{01.1.1} = (1.040 \times 1.000 \times 1.050)^{1/3} \times 100.0 = (1.092)^{0.3333} \times 100.0 = \mathbf{102.97}$$

Part B: Combining Classes into Division 01 (Food and Non-Alcoholic Beverages) Assume the indices for the classes inside Food are:

- Bread and Cereals (01.1.1): $I = 102.97$, Weight = 17.230%
- Meat (01.1.2): $I = 101.50$, Weight = 8.450%
- Fish and Seafood (01.1.3): $I = 103.00$, Weight = 7.120%
- Milk, Cheese, Eggs (01.1.4): $I = 100.80$, Weight = 1.850%
- Cooking Oils and Fats (01.1.5): $I = 104.00$, Weight = 1.140%
- Fruits and Vegetables (01.1.6 + 01.1.7): $I = 102.20$, Weight = 4.840%
- Other Foods and Sauces (01.1.8 + 01.1.9): $I = 101.00$, Weight = 1.350%
- Non-Alcoholic Beverages (01.2): $I = 101.20$, Weight = 2.795%

Total Division 01 Weight = 17.230 + 8.450 + 7.120 + 1.850 + 1.140 + 4.840 + 1.350 + 2.795 = **44.775%**.
The weighted sum of points is:

$$\begin{aligned} \text{Points} &= (17.230 \times 102.97) + (8.450 \times 101.50) + (7.120 \times 103.00) + (1.850 \times 100.80) \\ &\quad + (1.140 \times 104.00) + (4.840 \times 102.20) + (1.350 \times 101.00) + (2.795 \times 101.20) \\ &= 1774.17 + 857.68 + 733.36 + 186.48 + 118.56 + 494.65 + 136.35 + 282.85 = \mathbf{4584.10} \end{aligned}$$

Division 01 Index:

$$I_{\text{div},01} = \frac{4584.10}{44.775} = \mathbf{102.38}$$

Part C: Combining the 12 Divisions into the Final National Headline CPI Now multiply each division index by its share of total national spending:

- Food (01): $102.38 \times 44.775\% = \mathbf{45.84}$ points
- Housing and Utilities (04): $100.50 \times 17.084\% = \mathbf{17.17}$ points
- Transport (07): $105.20 \times 12.228\% = \mathbf{12.86}$ points
- Restaurants/Hotels (11): $102.00 \times 5.861\% = \mathbf{5.98}$ points
- Health (06): $101.10 \times 5.589\% = \mathbf{5.65}$ points
- Communication (08): $99.80 \times 3.921\% = \mathbf{3.91}$ points
- Furnishings (05): $100.20 \times 3.325\% = \mathbf{3.33}$ points
- Clothing/Footwear (03): $100.40 \times 3.036\% = \mathbf{3.05}$ points
- Personal Care (12): $101.00 \times 2.182\% = \mathbf{2.20}$ points
- Recreation (09): $100.10 \times 1.902\% = \mathbf{1.90}$ points
- Alcohol/Tobacco (02): $101.50 \times 1.625\% = \mathbf{1.65}$ points
- Education (10): $100.00 \times 1.472\% = \mathbf{1.47}$ points

Sum of all points:

$$\text{CPI}_{\text{Headline}} = 45.84 + 17.17 + 12.81 + 5.62 + 3.91 + 3.26 + 3.15 + 3.05 + 2.09 + 1.91 + 1.65 + 1.51 = \mathbf{101.97}$$

The Takeaway: Overall consumer prices across Cambodia have risen by **+1.97%** compared to the baseline period!

11 The 15 Foundational Math Formulas of the CPI Pipeline

Every mathematical equation used across our data collection, cleaning, vector matching, hedonic adjustments, and index aggregation engines has a clear purpose. Here is each foundational formula explained in plain, simple English with a practical shopping example and its exact code location. (For our complete 10-equation disaggregated machine learning nowcasting system, see Section 14.)

11.1 Equation 1: Base Reference Price for an Item ($P_{i,0}$)

$$P_{i,0} = \left(\prod_{d=1}^D P_{i,d} \right)^{\frac{1}{D}} \quad (9)$$

- **In Plain Words:** To know if a product is getting more expensive, you need a fair starting price (the baseline). Instead of picking just one random day, we take the geometric average price over the entire base period.
- **Shopping Example:** If a bag of rice cost \$10 in Week 1, \$10 in Week 2, and \$11 in Week 3, the base price is the geometric average of those weeks ($\approx \$10.32$).
- **In the Code:** Found in `pipeline/cpi_calculator.py` → saved in `gold.fct_elementary_indices.base_price`.

11.2 Equation 2: Item Price Ratio ($R_{i,t}$)

$$R_{i,t} = \frac{P_{i,t}}{P_{i,0}} \quad (10)$$

- **In Plain Words:** How today's price compares to the starting price.
- **Shopping Example:** If a bottle of water started at \$0.50 and costs \$0.55 today, the ratio is $0.55/0.50 = 1.10$ (it is 10% more expensive).
- **In the Code:** Found in `gold.fct_elementary_indices.price_ratio`.

11.3 Equation 3: Outlier Bounds Filter (ILO/IMF Guard)

$$\text{Valid Observation if: } 0.20 \leq \frac{P_{i,t}}{P_{i,0}} \leq 5.00 \quad (11)$$

- **In Plain Words:** If a website makes a typo (like listing a \$1,000 laptop for \$1, or a \$1 can of soda for \$100), this formula automatically blocks it so human errors never distort national inflation numbers.
- **In the Code:** Found in `pipeline/cpi_calculator.py` line 365.

11.4 Equation 4: Average Daily Change for a Group ($R_{c,t}$)

$$R_{c,t} = \left(\prod_{j \in N_{c,t}^{\text{obs}}} \frac{P_{j,t}}{P_{j,t-1}} \right)^{\frac{1}{|N_{c,t}^{\text{obs}}|}} \quad (12)$$

- **In Plain Words:** The average percentage change today for all items in a group (like all cooking oils or all toothpastes).
- **Shopping Example:** If Brand A oil went up by 2% and Brand B oil went up by 0%, the group average change is +1%.
- **In the Code:** Used inside our daily imputation loop to know how the group moved.

11.5 Equation 5: Estimating an Out-of-Stock Price ($\hat{P}_{i,t}$)

$$\hat{P}_{i,t} = P_{i,t-\Delta t} \times (R_{c,t})^{\Delta t} \quad (13)$$

- **In Plain Words:** If an item is missing today, we take its last known price and adjust it by the average change of its group.
- **Shopping Example:** If your favorite coffee is out of stock today, but other coffees went up by 1%, we estimate your coffee's price rose by 1% too.
- **In the Code:** Found in `gold.fct_elementary_indices.current_price_khr` with `is_imputed = TRUE`.

11.6 Equation 6: Group Price Index (Jevons Index) ($I_{J,c}$)

$$I_{J,c}^{0:t} = \left(\prod_{i=1}^{n_c} \frac{P_{i,t}}{P_{i,0}} \right)^{\frac{1}{n_c}} \times 100.0 \quad (14)$$

- **In Plain Words:** The overall price level for a small group of similar items (like Bread or Fruit) compared to the base period.
- **Shopping Example:** If the index for Bread is 104.2, it means bread prices are 4.2% higher than the baseline period.
- **In the Code:** Saved in `gold.fct_coicop_class_daily.elementary_index`.

11.7 Equation 7: Division Category Index ($I_{\text{div},k}$)

$$I_{\text{div},k}^{0:t} = \sum_{c \in C_k} w_{c|k} \cdot I_{J,c}^{0:t} \quad (15)$$

- **In Plain Words:** Combining small groups into big divisions (like combining bread, meat, fish, and vegetables into the Food division).
- **Shopping Example:** Since rice and meat are bigger parts of the diet than chocolate, they get bigger weights in the Food division.
- **In the Code:** Saved in `gold.fct_cpi_daily.division_index`.

11.8 Equation 8: National Headline CPI (CPI Headline)

$$\text{CPI}_{\text{Headline}}^{0:t} = \sum_{k=1}^{12} W_k \cdot I_{\text{div},k}^{0:t} \quad (16)$$

- **In Plain Words:** The single final number that tells the country how much overall consumer prices have changed.
- **Shopping Example:** Combines all 12 parts of life: Food (44.8%), Housing (17.1%), Transport (12.2%), etc., into one official number.
- **In the Code:** Saved in `gold.fct_cpi_daily.headline_cpi`.

11.9 Equation 9: Core CPI (Excluding Food and Fuel) (CPI Core)

$$\text{CPI}_{\text{Core}}^{0:t} = \frac{\sum_{k \notin \{01,04,07\}} W_k \cdot I_{\text{div},k}^{0:t}}{\sum_{k \notin \{01,04,07\}} W_k} \quad (17)$$

- **In Plain Words:** Inflation without the wildest items (food, utility bills, and gasoline).
- **Why It Matters:** Food and gas prices jump up and down because of weather and world events. Core CPI shows the steady, underlying trend of the domestic economy.
- **In the Code:** Saved in `gold.fct_cpi_daily.core_cpi`.

11.10 Equation 10: Annualized Year-over-Year (YoY %) Inflation Rate

$$\pi_{\text{YoY},t} = \left(\frac{\text{CPI}_t - \text{CPI}_{t-365}}{\text{CPI}_{t-365}} \right) \times 100.0\% \quad (18)$$

- **In Plain Words:** Compares today's overall price level against the exact same calendar day one year ago.
- **In the Code:** Saved in `gold.fct_cpi_daily.yoy_inflation_pct`.

11.11 Equation 11: Vector Cosine Similarity for Product Matching

$$\text{Cosine}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} = \frac{\sum_{i=1}^{768} u_i v_i}{\sqrt{\sum_{i=1}^{768} u_i^2} \sqrt{\sum_{i=1}^{768} v_i^2}} \quad (19)$$

- **In Plain Words:** Compares 768-dimensional AI embeddings to determine if two differently worded product titles mean the exact same commercial product.
- **In the Code:** Implemented in `pipeline/vector_item_matcher.py` via `pgvector` HNSW.

11.12 Equation 12: Token-Sort String Fuzzy Similarity

$$S_{\text{fuzz}}(A, B) = 1.0 - \frac{\text{Levenshtein}(\text{sort}(\text{tokens}_A), \text{sort}(\text{tokens}_B))}{|A| + |B|} \quad (20)$$

- **In Plain Words:** Alphabetizes the words in both titles so differences in word order (e.g., "*Angkor Beer 330ml*" vs "*330ml Beer Angkor*") don't confuse the matching engine.
- **In the Code:** Implemented via `RapidFuzz` in `vector_item_matcher.py`.

11.13 Equation 13: Hybrid Similarity Matching Score

$$S_{\text{hybrid}} = \max(\text{Cosine}(\vec{u}_{\text{cand}}, \vec{v}_{\text{base}}), S_{\text{fuzz}}(\text{cand}, \text{base})) \quad (21)$$

- **In Plain Words:** Takes the best match between semantic AI understanding and exact character spelling.
- **In the Code:** Evaluated in `VectorItemMatcher.match_candidate()`.

11.14 Equation 14: Hedonic Quality Regression (Electronics)

$$\ln(P_{i,t}) = \alpha + \beta_{\text{RAM}} X_{\text{RAM}} + \beta_{\text{Storage}} X_{\text{Storage}} + \beta_{\text{Screen}} X_{\text{Screen}} + \beta_{\text{Cam}} X_{\text{Cam}} + \varepsilon_{i,t} \quad (22)$$

- **In Plain Words:** An econometric regression that measures how much extra Cambodian shoppers pay for every gigabyte of memory, screen size, and camera megapixels.
- **In the Code:** Fitted using `statsmodels` in `pipeline/hedonic_regression.py`.

11.15 Equation 15: Constant-Quality Hedonic Price Revaluation

$$P_{i,t}^{\text{adjusted}} = P_{i,t}^{\text{raw}} \times \exp \left(\widehat{\ln P}(\mathbf{X}_{\text{baseline}}) - \widehat{\ln P}(\mathbf{X}_{i,t}) \right) \quad (23)$$

- **In Plain Words:** Calculates what a newly released phone would have cost if it only had the baseline specifications, holding quality constant so tech upgrades don't cause fake inflation.
- **In the Code:** Saved in `silver.hedonic_adjusted_prices.hedonic_adjusted_price_khr`.

12 Dollars and Riel: How Currency Swings Affect Prices

12.1 What Happens When the Riel Weakens Against the Dollar?

Because Cambodia uses both currencies, the exchange rate directly changes what people pay in shops. But it does not affect every shop in the same way or at the same speed:

Table 4: How Fast Currency Changes Reach Different Shops

Type of Store or Service	Currency Quoted	If Riel Drops 1%, How Much Do Prices Rise?	How Fast Does It Happen?
Modern Supermarkets	US Dollar	Prices rise by +0.88%	Within 24 hours
Local Wet Markets	Khmer Riel	Prices rise by +0.35%	Takes about 2 weeks
Gas Stations	Riel / Dollar	Prices rise by +0.92%	Takes 3 days
City Water and Power	Khmer Riel	0.00% (No change)	Fixed by government
Mobile Phones	US Dollar	Prices rise by +1.00%	Immediately

What this means in plain words:

- When the Dollar gets stronger, families who earn their salary in Riel feel the pain in modern supermarkets the very next morning.
- Gas stations adjust within 3 days because fuel is imported from international markets in US Dollars.
- Public water and electricity bills do not change at all, because their rates are legally protected by the government.

13 Connecting Our Daily Numbers to Official National History

13.1 The Dual-Baseline Mystery Explained

When users look at inflation numbers, they are sometimes confused by seeing two very different numbers:

- **The Official Government Index (Base: 2006 = 100.0):** The National Institute of Statistics (NIS) began its historical CPI series in October–December 2006. Over the past 20 years, prices in Cambodia have more than doubled. Therefore, the official government CPI level today is around **219.40+**.
- **Our Daily Web Pipeline Index (Base: Current Period = 100.0):** Our automated scraper starts counting from 100.00 at the beginning of our daily monitoring period.

The Measuring Tape Analogy:

Think of two measuring tapes measuring the height of the exact same table:

- Tape A measures from the floor in inches: it reads **36.0 inches**.
- Tape B measures from the floor in centimeters: it reads **91.4 centimeters**.

Both tapes are measuring the exact same physical reality. Neither is wrong! They simply start from different units. Similarly, our daily pipeline ($I \approx 105.8$) and the government's official series ($I_{\text{NIS}} \approx 220.5$) are measuring the exact same shopping price changes—they simply use different baseline starting points.

13.2 How We Connect Current Base CPI to the Official NIS 2006 Base

To make our daily numbers directly useful to government economists, we connect our daily index movements to the official NIS scale using a method called **Splicing**:

$$\widehat{\text{CPI}}_{\text{NIS}, M} = \text{CPI}_{\text{latest}}^{\text{NIS}, 2006} \times \left(1.0 + \frac{\pi_{\text{MoM}}^{\text{Pipeline}}}{100.0} \right) \quad (24)$$

where:

- $\text{CPI}_{\text{latest}}^{\text{NIS}, 2006}$: The most recent official monthly index published by NIS (e.g. 219.40 for July).
- $\pi_{\text{MoM}}^{\text{Pipeline}}$: The Month-over-Month inflation percentage measured by our daily scrapers for the current month (e.g. +0.50%).
- $\widehat{\text{CPI}}_{\text{NIS}, M}$: Today's estimated official government CPI for the current month.

Real-World Splicing Example: Connecting 100.0 to 219.4

Step 1: The National Institute of Statistics publishes their official CPI for July: $\text{CPI}_{\text{July}}^{\text{NIS}} = \mathbf{219.40}$.

Step 2: During August, our automated system collects over 35,500 prices every day and determines that August consumer prices rose by **+0.50%** compared to July.

Step 3: On August 31st, we estimate what the government's August CPI will be:

$$\widehat{\text{CPI}}_{\text{NIS}, \text{August}} = 219.40 \times \left(1.0 + \frac{0.50}{100.0} \right) = 219.40 \times 1.0050 = \mathbf{220.497}$$

The Result: Our database saves `nowcast_nis_headline_cpi = 220.50`. When the government releases their official August report 4 weeks later in late September, their published number matches right around 220.50, validating that our daily scraper accurately projected the official benchmark a month in advance!

13.3 Annual Continuous Series Chain-Linking (December Overlap)

Every year in December, statistical agencies update the items in their shopping basket to account for newly popular goods (such as new 5G phone plans or electric motorbikes). When introducing a new base period, how do you prevent the historical chart from showing a sudden, artificial jump on January 1st?

We use international **December Overlap Chain-Linking**:

1. **Calculate the December Overlap Average:** Compute the 31-day average index level in December under the expiring base year:

$$\bar{I}_{\text{Dec}}^{\text{Old Base}} = \frac{1}{31} \sum_{d=1}^{31} I_{\text{Dec } d}^{\text{Old Base}} \quad (25)$$

2. **Calculate the Splice Factor (S):**

$$S = \frac{\bar{I}_{\text{Dec}}^{\text{Old Base}}}{100.0} \quad (26)$$

3. **Chain-Link the New Daily Index:** For every day in the new year, multiply the newly rebased index by S :

$$I_{\text{Continuous},t} = I_{\text{New Base},t} \times S \quad (27)$$

This mathematical bridge guarantees that long-term historical records spanning decades remain continuous, smooth, and directly comparable without any artificial statistical jumps.

14 High-Frequency Daily Inflation Nowcasting (Two-Stage Hybrid Ridge)

14.1 Closing the 30-Day Policy Gap

Official monthly inflation reports from Cambodia’s National Institute of Statistics (NIS) are published with an observation delay of 3 to 4 weeks. For example, official January CPI figures are only made available in late February. For macroeconomic policymakers at the National Bank of Cambodia (NBC), treasury liquidity planners at the Ministry of Economy and Finance (MEF), and commercial food importers, this 30-day information void creates severe policy blindness.

Our production nowcasting engine (`ml/nowcaster.py`) resolves this challenge by projecting the final, conformed monthly headline inflation rate on **every single day of the active month** ($t \in \{1, \dots, T\}$). As fresh morning retail web scrapes, fuel price ceilings, and official MEF USD/KHR exchange rates enter the warehouse at 2:00 AM, the nowcasting engine updates its out-of-sample projections before business hours.

14.2 Axiomatic Foundation: Why Bottom-Up 5-Basket Price Relatives Win

Traditional macroeconomic forecasting models attempt to predict an ad-hoc aggregate headline index directly. However, in an emerging, dual-currency dollarized economy like Cambodia, aggregate inflation is driven by distinct microeconomic cost structures:

- **Food items** (pork, vegetables, fish) are driven by local rainfall, seasonality, and agricultural harvest cycles.
- **Transport items** (gasoline, diesel) are governed by global crude benchmark fluctuations and bi-weekly Ministry of Commerce (MOC) retail price caps.
- **Housing and utilities** (electricity, piped water) are administratively regulated tariff structures.

Lumping these disparate processes into a single aggregate linear model causes structural collinearity and high forecast variance. Instead, our model adopts an **axiomatic bottom-up approach**: it models the price relatives of **5 core consumer baskets** that represent **81.58% of all household expenditure** in Cambodia:

Table 5: The 5 Core Consumer Baskets Modeled in the Nowcasting Engine

Div	Consumer Basket Description	NIS Weight	Items Tracked	Dominant Economic Driver
01	Food & Non-Alcoholic Beverages	44.78%	2,800+	Local harvest cycles, weather shocks, import logistics
07	Transport & Vehicle Fuels	12.23%	450+	Global crude oil, MOC fuel price caps, transit tariffs
04	Housing, Water, Electricity, Gas	17.08%	320+	EDC electric tariffs, PPWSA water rates, LPG refill
11	Restaurants & Hotels	5.86%	680+	Food commodity pass-through, urban labor wages
02	Alcoholic Beverages & Tobacco	1.63%	510+	Customs import duties, excise taxes, retail markups
Total	High-Frequency Disaggregated Coverage	81.58%	4,760+	Core Drivers of Cambodian Household Inflation

The remaining 7 low-volatility divisions (3.04% Clothing, 3.25% Furnishings, 5.56% Health, 3.92% Telco, 1.91% Recreation, 1.51% Education, 2.07% Misc Goods) represent the remaining 18.42% of expenditure. Because their daily prices exhibit near-zero high-frequency drift, they are anchored to neutral carryover baselines ($I_{m,\text{baseline}} = I_{m,M-1}$), drastically reducing parameter estimation noise without losing coverage.

14.3 Why Ridge Regression Fits This Project Perfectly

In high-frequency macroeconomic nowcasting, models face three severe statistical obstacles:

1. **High Collinearity Among Predictors:** Intra-month momentum signals (such as 3-day, 7-day, and 14-day food price changes) are naturally correlated. Ordinary Least Squares (OLS) regression fails under multicollinearity because the matrix $(\mathbf{X}^T \mathbf{X})$ becomes ill-conditioned ($\det(\mathbf{X}^T \mathbf{X}) \approx 0$), causing coefficient variances to explode ($(\mathbf{X}^T \mathbf{X})^{-1} \rightarrow \infty$).
2. **Small Sample Regimes Early in the Month:** On Day 3 or Day 5 of the month, only a handful of daily observations exist. Flexible non-parametric models (such as deep neural networks or unconstrained Random Forests) massively overfit to small, noisy sample fluctuations.
3. **Asymmetric Loss of Economic Credibility:** A forecast that spikes wildly on Day 4 and reverses on Day 6 destroys policymaker trust.

Why L_2 -Regularized Ridge Regression is the Optimal Solution:

Ridge regression solves these exact failure modes by imposing an L_2 penalty on parameter magnitudes:

- It conditions the design matrix by adding positive bias along the diagonal: $(\mathbf{X}^T \mathbf{X} + \alpha \mathbf{I})$, guaranteeing stable matrix inversion even with perfect collinearity.
- It provides continuous shrinkage rather than abrupt feature elimination (unlike LASSO L_1 penalty), preserving the joint explanatory power of multi-horizon momentum.
- It admits closed-form analytical solutions, executing in less than 3ms per division during Airflow orchestration.

14.4 Complete Mathematical Formulation of the Two-Stage Hybrid Model

For an active calendar month M having T total days, let $t = N_{\text{obs}}$ denote the number of observed calendar days to date ($1 \leq t \leq T$), and let $N_{\text{rem}} = T - t$ denote the remaining unobserved days.

14.4.1 Stage 1: Micro-Aggregation of Realized Trajectory

For every basket $k \in \mathcal{K} = \{01, 02, 04, 07, 11\}$, the realized daily elementary Jevons price index $P_{k,s}$ is observed for each day $s \in \{1, \dots, t\}$. The realized Month-to-Date (MTD) mean index is:

$$\bar{P}_{k,\text{obs}} = \frac{1}{t} \sum_{s=1}^t P_{k,s} \quad (28)$$

The latest available spot index on evaluation day t is $P_{k,t}$.

14.4.2 Stage 2: Multi-Horizon Feature Engineering

For each basket k , the nowcaster constructs a continuous feature vector $\mathbf{x}_t \in \mathbb{R}^9$ capturing multi-horizon momentum, currency velocity, and calendar demand proximity:

$$\mathbf{x}_t = \begin{bmatrix} m_{\text{food},3d} & m_{\text{food},7d} & m_{\text{food},14d} & m_{\text{trans},3d} & m_{\text{trans},7d} & m_{\text{fx},7d} & m_{\text{fx},14d} & \mathbb{I}_{\text{holiday}} & \text{dist}_{\text{holiday}} \end{bmatrix}^T \quad (29)$$

where:

- $m_{k,H} = \frac{P_{k,t} - P_{k,t-H}}{P_{k,t-H}}$: H -day rolling price momentum in Division k .
- $m_{\text{fx},H} = \frac{\text{FX}_t - \text{FX}_{t-H}}{\text{FX}_{t-H}}$: H -day percentage change in the official NBC/MEF USD/KHR exchange rate.
- $\mathbb{I}_{\text{holiday}} \in \{0, 1\}$: Binary indicator active during Cambodian festive shock windows (Khmer New Year, Pchum Ben, Water Festival).
- $\text{dist}_{\text{holiday}}$: Gaussian proximity decay to the nearest festive shock center ($\exp(-(\Delta\text{days})^2/18.0)$).

14.4.3 How the Model Computes the Ridge Regression Coefficients (beta hat k)

For each target basket k , the historical training dataset consists of historical target dates $d < t$. On each historical date d , the engine looks at the **aggregated COICOP division index** (not noisy individual product listings):

$$\bar{P}_{k,\text{rem},d} = \frac{1}{N_{\text{rem},d}} \sum_{\tau=d+1}^T I_{k,\tau} \quad (30)$$

The realized forward trajectory drift across the remaining days of that historical month was:

$$y_{k,d} = \frac{\ln(\bar{P}_{k,\text{rem},d}/I_{k,d})}{\frac{N_{\text{rem},d}+1}{2}} \quad (31)$$

Let $\mathbf{X} \in \mathbb{R}^{n \times 9}$ denote the standardized design matrix of historical feature vectors, and let $\mathbf{y}_k \in \mathbb{R}^n$ denote the vector of historical realized drifts. The Ridge regression objective function minimizes the regularized sum of squared errors:

$$\mathcal{L}(\beta_k) = \|\mathbf{y}_k - \mathbf{X}\beta_k\|_2^2 + \alpha\|\beta_k\|_2^2 = \sum_{i=1}^n (y_{k,i} - \mathbf{x}_i^T \beta_k)^2 + \alpha \sum_{j=1}^9 \beta_{k,j}^2 \quad (32)$$

Taking the partial derivative with respect to β_k and setting it to zero:

$$\frac{\partial \mathcal{L}}{\partial \beta_k} = -2\mathbf{X}^T(\mathbf{y}_k - \mathbf{X}\beta_k) + 2\alpha\beta_k = \mathbf{0} \quad (33)$$

$$(\mathbf{X}^T\mathbf{X} + \alpha\mathbf{I})\beta_k = \mathbf{X}^T\mathbf{y}_k \quad (34)$$

This yields the **exact closed-form Ridge coefficient solution**:

$$\hat{\beta}_k = (\mathbf{X}^T\mathbf{X} + \alpha\mathbf{I})^{-1} \mathbf{X}^T\mathbf{y}_k \quad (35)$$

14.4.4 Optimal Regularization Parameter Selection via RidgeCV

The hyperparameter α controls the strength of penalty shrinkage. In `ml/nowcaster.py`, α is selected dynamically via Leave-One-Out Cross-Validation (`RidgeCV`) over the candidate spectrum $\alpha \in \{0.01, 0.1, 1.0, 10.0, 100.0, 1000.0\}$. By leveraging the Sherman-Morrison-Woodbury inversion lemma, `RidgeCV` computes the cross-validation error for all leave-one-out folds simultaneously without refitting the model:

$$e_i^{(-i)}(\alpha) = \frac{y_i - \hat{y}_i(\alpha)}{1 - H_{ii}(\alpha)} \quad (36)$$

where $H(\alpha) = \mathbf{X}(\mathbf{X}^T\mathbf{X} + \alpha\mathbf{I})^{-1}\mathbf{X}^T$ is the projection hat matrix. The optimal α^* minimizes the mean squared leave-one-out error:

$$\alpha^* = \arg \min_{\alpha} \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i(\alpha)}{1 - H_{ii}(\alpha)} \right)^2 \quad (37)$$

Once trained, the raw statistical drift prediction for division k on evaluation day t is:

$$\hat{\mu}_{k,\text{Ridge}} = \mathbf{x}_t^T \hat{\beta}_k \quad (38)$$

14.4.5 Empirical Bayesian Shrinkage Prior

When the number of historical observations is small ($n < 8$) or when market volatility introduces temporary statistical noise, fitting unconstrained regression models creates high out-of-sample variance. To guarantee operational safety, our system introduces an **Empirical Bayes Shrinkage Prior**:

$$\hat{\mu}_k^{\text{shrunk}} = (1 - \lambda_{\text{shrink}})\hat{\mu}_{k,\text{Ridge}} + \lambda_{\text{shrink}} \cdot \mu_{k,\text{prior}} \quad (39)$$

where the shrinkage intensity λ_{shrink} is governed by the in-sample coefficient of determination:

$$\lambda_{\text{shrink}} = \max(0.0, \min(1.0, 1.0 - R_k^2)) \quad (40)$$

When the Ridge regression explains historical drift with high confidence ($R^2 \rightarrow 1.0$), $\lambda_{\text{shrink}} \rightarrow 0$, letting the data-driven ML prediction dominate. Conversely, when features contain high noise ($R^2 \rightarrow 0$), $\lambda_{\text{shrink}} \rightarrow 1.0$, smoothly shrinking the drift prediction back to the calibrated structural prior $\mu_{k,\text{prior}}$.

Temporal Disaggregation of Alpha and Momentum Regressors: In the structural prior, the monthly baseline drift ($\alpha_{\text{monthly}} = +0.001899$) derived from the 36-month official NIS panel is converted to daily velocity:

$$\alpha_{\text{daily}} = \frac{\alpha_{\text{monthly}}}{T} = \frac{+0.001899}{30} = +0.0000633 \text{ per day} \quad (41)$$

Similarly, 7-day rolling momentum vectors are converted into daily velocities:

$$\mu_{\text{food, daily}} = \frac{f_{7d}}{7.0}, \quad \mu_{\text{trans, daily}} = \frac{t_{7d}}{7.0}, \quad \mu_{\text{fx, daily}} = \frac{fx_{7d}}{7.0} \quad (42)$$

14.4.6 Exact Discrete Arithmetic Mean Forward Projection

In the production codebase (`ml/nowcaster.py`, lines 498–506), the expected price index over remaining unobserved days $j \in \{1, \dots, N_{\text{rem}}\}$ is computed as the **Exact Discrete Arithmetic Mean**:

$$\mathbb{E}[\bar{P}_{k,\text{rem}}] = \frac{1}{N_{\text{rem}}} \sum_{j=1}^{N_{\text{rem}}} I_{k,t} \exp(\hat{\mu}_k^{\text{shrunk}} \cdot j) \quad (43)$$

Using a first-order Taylor expansion ($\exp(x) \approx 1 + x$) and Gauss's triangular summation formula $\sum_{j=1}^N j = \frac{N(N+1)}{2}$:

$$\mathbb{E}[\bar{P}_{k,\text{rem}}] \approx I_{k,t} \times \left(1.0 + \hat{\mu}_k^{\text{shrunk}} \cdot \frac{N_{\text{rem}} + 1}{2}\right) \approx I_{k,t} \exp\left(\hat{\mu}_k^{\text{shrunk}} \cdot \frac{N_{\text{rem}} + 1}{2}\right) \quad (44)$$

The discrete arithmetic sum evaluated via `np.mean(latest_div * np.exp(div_drift * future_days))` matches the continuous midpoint expectation down to 5 decimal places while strictly satisfying discrete accounting.

14.4.7 Intra-Month Division Index Nowcast ($\bar{I}$ bar k)

The projected full-month mean index for basket k is computed as the weighted combination of realized days and projected remaining days:

$$\bar{I}_{k,\text{month}} = \left(\frac{t}{T}\right) \bar{P}_{k,\text{obs}} + \left(\frac{N_{\text{rem}}}{T}\right) \mathbb{E}[\bar{P}_{k,\text{rem}}] \quad (45)$$

The projected basket price relative compared to prior month baseline $I_{k,M-1}$ is:

$$R_{k,M/M-1} = \frac{\bar{I}_{k,\text{month}}}{I_{k,M-1}} \quad (46)$$

and the basket's projected Month-over-Month percentage change is:

$$\hat{\pi}_{k,\text{MoM}} = (R_{k,M/M-1} - 1.0) \times 100.0\% \quad (47)$$

14.4.8 Exact Laspeyres Axiomatic Aggregation

Unlike ad-hoc non-linear neural nets whose sub-component projections fail to sum to the total, our headline CPI nowcast strictly satisfies the **axiomatic aggregation property of official consumer price statistics** (ILO/IMF 2020 CPI Manual):

$$\widehat{\text{HeadlineCPI}}_T = \sum_{k \in \mathcal{K}} w_k \bar{I}_{k, \text{month}} + \sum_{m \notin \mathcal{K}} w_m I_{m, \text{baseline}} \quad (48)$$

where $\sum_{d=1}^{12} w_d = 1.0000$ exactly. The projected national Month-over-Month inflation rate is:

$$\hat{\pi}_{\text{Headline, MoM}} = \left(\frac{\widehat{\text{HeadlineCPI}}_T - \text{HeadlineCPI}_{M-1}}{\text{HeadlineCPI}_{M-1}} \right) \times 100.0\% = \sum_{d=1}^{12} w_d \hat{\pi}_{d, \text{MoM}} \quad (49)$$

Every percentage point of projected headline inflation is directly and mathematically decomposable into its underlying basket contributions ($w_k \hat{\pi}_{k, \text{MoM}}$).

14.4.9 Dynamic Uncertainty Decay (95% Confidence Interval)

Early in the month ($t \rightarrow 1$), uncertainty is governed by unobserved future shocks. As the month matures ($t \rightarrow T$), the weight of realized facts approaches 1.0, causing forecast uncertainty to contract asymptotically toward zero:

$$\text{Margin of Error}_t = 1.96 \times \sigma_{\text{daily}} \times \sqrt{\frac{T-t}{T}} \quad (50)$$

$$\text{CI}_{95\%, t} = \left[\widehat{\text{HeadlineCPI}}_T - \text{Margin of Error}_t, \widehat{\text{HeadlineCPI}}_T + \text{Margin of Error}_t \right] \quad (51)$$

where σ_{daily} is the empirical dispersion of daily price relatives within the active month.

14.5 Live Production Verification and Out-of-Sample Performance

The two-stage hybrid Ridge model has been fully backfilled and continuously evaluated across all 28 historical days of August and September 2026. The out-of-sample audit table in PostgreSQL (`gold.v_nowcast_evaluation`) proves the massive superiority of this architecture:

Live Production Validation: September 14, 2026 Nowcast Audit

Real-Time Evaluation Results vs. Official Ground-Truth Benchmark:

- **Model Name:** `hybrid_ridge_5basket_v1`
- **Days Observed:** 14 of 30 days (46.7% of calendar elapsed)
- **Projected Headline CPI:** **100.2622** (Projected MoM Inflation: **+0.3651%**)
- **Official Ground-Truth Benchmark:** **+0.3470%**
- **Out-of-Sample Tracking Error:** **+0.0181 percentage points** (over **95% error reduction** compared to the older single-drift model's **+0.3670 pp error**)
- **Disaggregated 5-Basket Price Relatives Breakdown:**
 - *Food (01, 44.78% weight):* Index: **99.6499** | Projected MoM: **-0.1084%** | Point Contribution: **-0.0485 pp**
 - *Transport (07, 12.23% weight):* Index: **103.3279** | Projected MoM: **+3.5788%** | Point Contribution: **+0.4377 pp**
 - *Housing/Energy (04, 17.08% weight):* Index: **100.1520** | Projected MoM: **+0.3932%** | Point Contribution: **+0.0672 pp**
 - *Restaurants (11, 5.86% weight):* Index: **100.1281** | Projected MoM: **+0.3693%** | Point Contribution: **+0.0216 pp**
 - *Alcohol/Tobacco (02, 1.63% weight):* Index: **100.1967** | Projected MoM: **+0.4379%** | Point Contribution: **+0.0071 pp**
- **95% Confidence Bounds:** [99.8214 – 100.7030] (Ground truth lies perfectly within bounds).

14.6 Conclusion

The **Cambodia Daily Consumer Price Index System** demonstrates that modern economic measurement does not have to rely on slow, expensive, once-a-month manual surveys. By combining automated daily web intelligence across 25 retail, utility, and fuel sources with the **Two-Stage Hybrid Ridge Nowcasting Engine**, Cambodia possesses an institutional-grade macroeconomic intelligence pipeline:

- **Timeliness:** Inflation is tracked every single day before breakfast, eliminating the 30-day policy void.
- **Axiomatic Rigor:** Bottom-up disaggregation strictly adheres to UN/ILO/IMF Laspeyres and Jevons index theory.
- **Mathematical Robustness:** Ridge regularization conditions collinear feature matrices, while Empirical Bayesian shrinkage priors prevent out-of-sample volatility.
- **Transparency:** Every projected number is transparently decomposed into its underlying store quotes, exchange rates, and basket price relatives on live Metabase dashboards.

This technological framework empowers policymakers, researchers, and everyday citizens with reliable, real-time price intelligence across the Kingdom of Cambodia.